

2021-2022 HEU Algorithm Society Singles Day Challenge Contest

Contest Session

November 11, 2021

打开B站搜索并关注“嘉然今天吃什么”([传送门](#))。关注嘉然然，顿顿解馋馋!!!

A. *Diana* 的糖果

B. 不许说“捏”

C. 受到制裁的群主

D. 最甜甜甜甜的小草莓

E. *ybb* 的 01串

F. 查询 *Ava* 闲的程度

G. *Nanami* 想要 Flip

H. 勇敢牛牛

I. *ybb* 小火车

J. 和 *Diana* 吃烧烤

K. 欢迎来到 *Gensokyo*

L. *Yukari* 修建结界

M. *Flandre* 的简单题

N. *Reimu* 的城管任务

O. *Marisa* 的魔法棋盘

A. Diana 的糖果



Diana 早上起床后第一件事是找她喜欢的糖果吃。*Diana* 有 N 颗糖果锁在罐子中。每颗糖果有一个仅由数字组成编号，而锁的密码是**所有糖果编号的最长公共前缀的长度**。*Diana* 忘记了密码又想吃糖果，所以请你通过糖果编号帮她找回密码。

编号的前缀指编号从第一个数字一直连续到后面某一个数字组成的一个子字符串。例如：“10032”的前缀有“1”、“10”、“100”、“1003”、“10032”，但是“1002”、“14”等就不是它的前缀。

公共前缀指若干个编号拥有的相同的前缀。例如“010414”、“0104773”、“010114514”的公共前缀有“0”、“01”、“010”，而“10”、“773”、“0104”，就不是它们的公共前缀。而最长公共前缀指的是在所有的公共前缀中长度最长的那个，在这里即为“010”，她的长度为 3。

Input

第一行包含一个整数 N ($2 \leq N \leq 30000$) 表示糖果数量。

接下来 N 行，每行包含一个糖果编号 s_i ($1 \leq |s_i| \leq 20$)。

Output

输出一行一个数字，表示 n 个编号的最长公共前缀长度。

Example

Input

```
4
00209
00219
00999
00909
```

Output

2

Input

2
1
2

Output

0

Input

3
77012345678999999999
77012345678901234567
77012345678998765432

Output

12

Note

对于第一个样例，四个糖果编号的所有公共前缀为“0”、“00”，最长为“00”，因此答案为 2。
对于第二个样例，两个糖果编号没有公共前缀，因此答案为 0。

B. 不许说“捏”



Diana 喜欢在 QQ 群小团体上发言。原来的小团体里面是可以自由发言的，但是不知道哪一天起，群主制定了一个群规，禁止群里发“捏”，违者禁言。禁言规则是在接下去 N 天中，第 i 天发“捏”的人禁言 a_i 秒，而 Diana 会在每天的第 0 秒发一个带“捏”的句子，然后被禁言。Diana 想尽快提升她的群等级，因此没有被禁言的时间她都在进群。现在 Diana 想知道第几天她的累计进群时间达到 t 秒，请你帮她算一算。

Input

第一行包含两个整数 N ($1 \leq N \leq 100$)、 t ($1 \leq t \leq 10^6$)，分别表示禁言规则持续时间和 Diana 想达成的累计进群时间；

第二行包含 N 个整数 $a_1, a_2, a_3, \dots, a_n$ ，其中 a_i ($0 \leq a_i \leq 86400$) 表示第 i 天发“捏”的禁言时间。

输入保证 n 天内 Diana 能完成进群目标。

Output

输出一行一个整数 x ，代表 Diana 最早在第 x 天达成进群目标。

Example

Input

```
2 2
86400 86398
```

Output

```
2
```

Input

```
2 86400
0 86400
```

Output

```
1
```

Note

一天有 $60 \times 60 \times 24 = 86400$ 秒。

对于第一个样例，Diana 第一天被禁言 86400 秒，累计进群 0 秒；Diana 第二天被禁言 86398 秒，累计进群 2 秒，达成目标。

对于第二个样例，Diana 第一天被禁言 0 秒，累计进群 86400 秒，达成目标。

C. 受到制裁的群主



Diana 在群里被禁言的事情很快被 *Bella* 知道了。*Bella* 想要教训一下欺负 *Diana* 的群主，而群主听到了风声后就马上躲到了 贡橙大学 11 号楼的某一间教室里。

11 号楼有 N 间从 1 编号到 n 教室，这些教室可以看成分布在一条直线上。*Bella* 在直线上从左往右跑过时观察到：先是所有编号为奇数的教室从小到大排列成一排，然后所有编号为偶数的教室从小到大排列成一排拼接在后面。更具体地说，当 N 为偶数的时候，教室按照这个规则排序之后为 $\{1, 3, 5, \dots, n-1, 2, 4, 6, \dots, n\}$ ；当 N 为奇数的时候，教室按照这个规则排序之后为 $\{1, 3, 5, \dots, n, 2, 4, 6, \dots, n-1\}$ 。

据群友透露，群主藏在直线上从左往右数的第 k 间教室里。由于 *Bella* 不太会数数，所以请你告诉她群主藏身的教室的编号，以便让 *Bella* 找到群主。

Input

一行包含两个整数 N ($1 \leq N \leq 10^{12}$)、 k ($1 \leq k \leq N$) 分别表示教室的数量和群主藏身的教室在直线上从左往右数第几间。

Output

输出一行一个数字，代表群主藏身的教室的编号。

Example

Input

10 3

Output

5

Input

7 7

Output

6

Note

对于第一个样例，教室在直线上的分布为 $\{1, 3, 5, 7, 9, 2, 4, 6, 8, 10\}$ ，从左往右数第 3 间教室编号为 5，群主藏在 5 号教室，因此输出 5。

对于第二个样例，教室在直线上的分布为 $\{1, 3, 5, 2, 4, 6\}$ ，从左往右数第 7 间教室编号为 6，群主藏在 6 号教室，因此输出 6。

Ⅱ. 最甜甜甜甜的小草莓



群主被 *Bella* 教训后决定向 *Diana* 道歉，于是他去买了一些最甜甜甜甜的小草莓送给了 *Diana*。

Diana 收到了 N 颗草莓，其中每一颗草莓的重量为 w_i 。*Diana* 想分一半重量的草莓给 *Bella* 酱，她想知道在不切割草莓的前提下能不能将**恰好**一半重量的草莓分给 *Bella* 酱（因为如果不能完美均分 *Diana* 会把草莓全部吃掉，嘿嘿）。为了解决这个问题，*Diana* 去请教了 *Ava* 酱，可是 *Ava* 酱正在玩游戏不想理她，因此这个问题就交给你解决了。

Input

第一行包含一个整数 N ($1 \leq N \leq 100$)，表示草莓的数量。

第二行包含 N 个整数 $w_1, w_2, w_3, \dots, w_n$ ，其中 w_i ($w_i = 100$ 或 200) 表示第 i 个草莓的重量。

Output

输出一个 “YES” 或 “NO” （不含引号）表示是否能将草莓分恰好一半重量给 *Bella*。

Example

Input

3
100 200 100

Output

YES

Input

4
100 100 100 200

Output

NO

Note

在第一个样例中 *Diana* 可以选择将两个重 100 的草莓或一个重 200 的草莓送给 *Bella* 。

在第二个样例中可以证明不可能在不切割草莓的情况下分恰好一半重量的草莓给 *Bella* 。

E. ybb 的 01串



没有理睬 *Diana* 的 *Ava* 正在网上和 *JXT* 玩一个和有关的 01 串游戏。

对于一个长度为 N 的 01 串 s ，如果存在子串“101”，那么 *JXT* 就会认为这个 01 串是 *ybb* 的。例如：“101100”、“011010”等 01 串是 *ybb* 的，而“010”、“010010”等 01 串不是 *ybb* 的。由于 *ybb* 的 01 串不会有什么好下场。01 串 s 想将自己身上的一些“1”变成“0”，从而使得自己不是 *ybb* 的。那么它最少要替换掉几个 1 呢？

01 串是指仅包含 0 和 1 的字符串。

一个字符串的子串指的是，在字符串的前面和后面连续删除若干字符（可以不删）后留下的字符串。例如“010”的子串有“0”、“1”、“010”、“10”等，而“00”、“11”、“0101”、“110”等不是它的子串。

Input

第一行包含一个整数 N ($3 \leq N \leq 100$)，表示 01 串的长度。

第二行包含 N 个用空格隔开的字符 $s_1, s_2, s_3, \dots, s_n$ ，其中 s_i ($s_i = \text{“0”}$ 或 “1”) 表示 s 第 i 个位置的字符。

Output

输出一个整数，表示最少需要替换的“1”的数量。

Example

Input

```
10
1 1 0 1 1 0 1 0 1 0
```

Output

```
2
```

Input

```
5
1 1 0 0 0
```

Output

```
0
```

Input

```
4
1 1 1 1
```


Output

0

Note

对于第一个样例，将 s_2 和 s_7 替换为“0”即可使 s 不是 ybb 的，也可将 s_4 和 s_7 替换为“0”使 s 不是 ybb 的。可以证明只替换一个字符不能使 s 不是 ybb 的，因此输出 2。

对于第二、三个样例，由于初始 s 不是 ybb 的，不用替换字符，因此输出 0。

F. 查询 *Ava* 闲的程度



Ava 最近很闲，*Ava* 和 *Nanami* 要从长度为 N 的 01 串 s 中各自选择一个长度大于等于 $\lfloor \frac{N}{2} \rfloor$ 的子串（ $\lfloor x \rfloor$ 表示小于等于 x 的最大整数。例如 $\lfloor 3.5 \rfloor = 3$ ， $\lfloor 4 \rfloor = 4$ ）。选择的两个子串的区域**不能完全相等**。

Nanami 先选子串 t ，*Ava* 后选子串 w 。每次 *Ava* 都故意使 $f(w)$ 是 $f(t)$ 的整数倍（ $f(x)$ 是将 01 串转化为二进制的函数，例如 $f("010") = (10)_2 = (2)_{10}$ ），这让 *Nanami* 觉得 *Ava* 很闲。于是她找到你，给了你她和 *Ava* 玩的 T 局游戏的 01 串，要求你说出这 T 局游戏里她们分别选了那些子串。因为 *Nanami* 其实自己也记不清她们选了哪些子串，所以对于每局游戏，你只要描述**任意满足条件的**两个区间断点不全相等子串 w 、 t ，使得 $f(w)$ 是 $f(t)$ 的整数倍就可以了。**可以被证明一定能找到答案去应对 *Nanami*。**

Input

第一行包含一个整数 T ($1 \leq T \leq 10^3$)，代表游戏的局数。

接下来每局游戏包含两行：

第一行包含一个整数 N ($2 \leq N \leq 2 \times 10^4$)，代表这局游戏的 01 串的长度；

第二行包含一个字符串 s ，代表这局游戏的 01 串 s 。

输入数据保证 $\sum N < 10^5$ 。

Output

对于每一局游戏输出一行包含 4 个整数 l_1 、 r_1 、 l_2 、 r_2 。

令 s 第一个字符下标为 1， w 是 s 下标从 l_1 连续到 r_1 的子串， t 是 s 下标从 l_2 连续到 r_2 的子串，输出要保证 $f(w)$ 是 $f(t)$ 的整数倍，并且 (l_1, r_1) 与 (l_2, r_2) 不能完全相同，还要满足 $|w| \geq \lfloor \frac{n}{2} \rfloor$ 且 $|t| \geq \lfloor \frac{n}{2} \rfloor$ ， $|t|$ 和 $|w|$ 分别代表字符串 t 和字符串 w 的长度。如果答案有多个，输出任意一个满足要求的答案即可。

Example

Input

```
7
6
101111
9
111000111
8
10000000
5
11011
6
001111
3
101
30
100000000000000100000000000000
```

Output

```
3 6 1 3
1 9 4 9
5 8 1 4
1 5 3 5
1 6 2 4
1 2 2 3
1 15 16 30
```

Note

对于第一局游戏 $f(w) = f(1111) = 15$ ， $f(t) = f(101) = 5$ ；

对于第二局游戏 $f(w) = f(111000111) = 455$ ， $f(t) = f(000111) = 7$ ；

对于第三局游戏 $f(w) = f(0000) = 0$ ， $f(t) = f(1000) = 8$ ；

对于第四局游戏 $f(w) = f(11011) = 27$ ， $f(t) = f(011) = 3$ ；

对于第五局游戏 $f(w) = f(001111) = 15$ ， $f(t) = f(011) = 3$ ；

G. Nanami 想要 Flip



Nanami 某一天想要玩 Flip 游戏，于是她找到了 Ava。Nanami 和 Ava 玩的游戏规则如下，取一个 01 串 s ， s 是回文串，双方轮流进行一下操作之一：

1. 修改 s 中一个“0”为“1”，消耗 1 点体力；
2. 若上一个人没有翻转 s 且 s 不是回文串，翻转 s ，不消耗体力；否则不能进行此操作。

当 s 全为“1”时游戏结束，消耗体力少的玩家获胜，若玩家消耗体力一样，则是平局。现在 Nanami 是先手，Ava 是后手，两个人都采取最佳策略玩了 T 局游戏，求最终游戏结果。

翻转一个字符串指将字符串从右往左读形成一个新字符串，如“010010”翻转后是“010010”，“01001110”翻转后是“01110010”。若一个字符串翻转后和原来相同，则它是回文串，如“010010”是回文串，“01001110”不是回文串。

Input

第一行包含一个整数 T ($1 \leq T \leq 10^3$)，代表游戏的局数。

接下来每局游戏包含两行：

第一行包含一个整数 N ($1 \leq N \leq 10^3$)，代表这局游戏的 01 串的长度；

第二行包含一个字符串 s ，代表这局游戏的 01 串 s 。

Output

对于每一场比赛，输出一行字符串（不含引号）

如果 Nanami 赢了，输出“ALICE”；

如果 Ava 赢了，输出“BOB”；

如果平局，输出“DRAW”。

Example

Input

```
2
4
1001
1
0
```

Output

```
BOB
BOB
```

Note

对于第一局游戏：第一回合 *Nanami* 只能进行操作 1，第二回合 *Ava* 翻转 s ，第三回合 *Nanami* 只能进行操作 1，游戏结束。*Nanami* 消耗 2 点体力，*Ava* 消耗 0 点体力。*Ava* 获胜，输出“BOB”；

对于第二局游戏：第一回合 *Nanami* 只能进行操作 1，游戏结束。*Nanami* 消耗 1 点体力，*Ava* 消耗 0 点体力。*Ava* 获胜，输出“BOB”。

III. 勇敢牛牛



不出所料，*Nanami* 在和 *Ava* 的游戏中输多赢少，于是她准备开 *ybb* 小火车出去转转。但是小火车在启动前需要“勇敢牛牛”之力。“勇敢牛牛”之力越多，小火车就越不怕困难，也就开得越远；因此 *Nanami* 找到了 *Bella*。*Bella* 拉来了 N 头牛，每头牛有一个牛牛值。如果两头牛的牛牛值按位与（AND）结果为 0，则两头牛是亲密的。小火车可按照如下规则获得“勇敢牛牛”之力：

- *Bella* 命令一头牛主动登上小火车，不获得“勇敢牛牛”之力；
- *Bella* 命令一头上车的牛 A 将和自己亲密的另一头牛 B 吸引上火车，小火车获得等同于牛 A 的牛牛值的“勇敢牛牛”之力；
- 每头牛只能上车一次，并且只能被一头牛吸引上车；但是一头牛可以多次吸引一头牛上火车。

牛会无条件听从 *Bella* 的命令，*Bella* 当然算不出小火车最多获得多少“勇敢牛牛”之力，所以请你帮她算。

按位与（AND）指的是将十进制数转化为二进制数后再按照二进制的每一个数位做与运算（其中一个数超过最高位的数位取 0）。例如： $(11)_{10}$ and $(7)_{10} = (1011)_2$ and $(0111)_2 = (0011)_2 = (3)_{10}$ 。更详细的解释见 [与运算](#)。

Input

第一行包含一个整数 N ($1 \leq N \leq 2 \times 10^5$)，表示牛的数量；
第二行包含 N 个整数 $a_1, a_2, a_3, \dots, a_n$ ，其中 a_i ($0 \leq a_i \leq 2 \times 10^5$) 表示第 i 只牛的牛牛值。

Output

输出一行包含一个整数，表示小火车获得最高的“勇敢牛牛”之力的值。

Example

Input

```
3
1 2 3
```

Output

```
2
```

Note

第二头牛主动上车后，再吸引第一头牛上车，获得 2 点“勇敢牛牛”之力。这时第三头牛无法吸引其他牛，也无法被其他牛吸引，所以主动上车，不获得“勇敢牛牛”之力。小火车最终获得值为 2 的“勇敢牛牛”之力。可以证明其他方案无法获得更多的“勇敢牛牛”之力。

II. ybb 小火车



有病吧

由于牛牛上了火车之后到处乱跑，*Nanami* 决定将它们都关在笼子里。火车上有 N 头牛，第 i 头牛的体积为 a_i ，*Nanami* 有 M 个笼子，每个笼子的体积都是 K ，现在 *Nanami* 按照以下规则将牛牛抓到笼子里：

1. 笼子的剩余体积大于等于牛的体积即可将牛抓到笼子里；
2. 牛被扔下火车后就不在笼子里了，牛被扔下火车后可能使一个非空的笼子变成空笼子；
3. *Nanami* 在最开始会选择一个空笼子作为当前手上的笼子，**只能将牛关进 *Nanami* 手上的笼子里**。*Nanami* 在**任意时刻**都可以将手上的笼子更换为一个空笼子，**但不能更换为非空的笼子**；*Nanami* 在任何时刻手上都只有一个笼子；
4. *Nanami* 在**任意时刻**都能将当前所有关在笼子的牛中那一头**最先进入**笼子的牛扔下火车；

Nanami **按顺序**从第 1 只牛开始到第 n 只牛结束，对于 *Nanami* 在抓第 i 只牛时，她可以利用上述 4 个规则操作将牛关入笼子中（可以多次使用 3、4 规则），必须保证把第 i 头牛关进笼子。（注：在把第 i 头牛抓到笼子**以后**，*Nanami* 可以利用 4 规则把第 i 头牛扔下火车）

Nanami 发现**完成上述所有操作后**，火车上的牛牛就全部都在关在笼子里了。

Nanami 知道**完成上述所有操作后**，火车上最多有多少牛牛，但她还是问了你，真的绝绝子！

提示：不理解题意可以看一下样例解释。

Input

第一行包含三个整数 N 、 M 、 K ($1 \leq N, M \leq 2 \times 10^5, 1 \leq K \leq 10^9$) 分别代表牛的数量，笼子的数量和单个笼子的体积；

第二行包含 N 个整数 $a_1, a_2, a_3, \dots, a_n$ ，其中 a_i ($1 \leq a_i \leq K$) 代表第 i 头牛的体积。

output

一行包含一个整数，代表**完成上述所有操作后**火车上最多有多少牛牛。

Example

Input

5 2 6
5 2 1 4 2

Output

4

Input

5 1 4
4 2 3 4 1

Output

1

Input

5 3 3
1 2 3 1 1

Output

5

Note

对于第一个样例，*Nanami* 最佳方案是先拿空笼子关住第 1 头牛，然后再换上空笼子关住第 2、3 头牛，之后她释放第 1 头牛(此时不能释放第 2 头牛，因为第 1 头牛比它更早进入笼子里)，再得到空笼子关住第 4、5 头牛，最后火车上一共有 4 头牛。

有一种合法方案是，拿一个空笼子先关进第 1 头牛，然后换一个笼子关进第 2、3 头牛，之后释放第 1 头牛，得到一个空笼子，再释放第 2 头牛，使手上笼子剩余体积增加，然后关上第 4 头牛；最后换一个空笼子关第 5 头牛，最后火车上一共有 3 头牛，不是最佳的方案。

还有一种合法方案是，先抓住第 1 头牛，然后放了，再抓住第 2 头牛，然后放了，再抓住第 3 头牛，然后放了，再抓住第 4 头牛，然后放了，最后抓住第 5 头牛，然后放了。最后火车上一共有 0 只牛，不是最优方案。

对于第二个样例，*Nanami* 最佳方案是前 4 头牛关住后就立刻释放，最后关住第 5 头牛，最后火车上一共有 1 头牛。

对于第三个样例，*Nanami* 最佳方案是先拿空笼子关住第 1、2 头牛，然后再换上空笼子关住第 3 头牛，最后再换上空笼子关住第 4、5 头牛。最后火车上一共有 5 头牛。

注：释放牛、把牛放了都是把牛扔下火车的意思。

Ⅱ. 和 Diana 吃烧烤



Nanami 用 ybb 小火车将一车牛牛拉去卖了之后拿到了一些钱。她拿着这些钱去和 Diana 吃烧烤。她们一共买了 N 份烧烤食品将食品并按照顺序围成了一个圈，其中每一份食品的辣度为正整数 a_i 。Nanami 为了让 Diana 辣得说不出话来，她让 Diana 按照一下规则吃烧烤：

Diana 将食品 1 到 N 依次编号，并按食品编号的顺序将食品摆成一个圆，然后从 1 号食品往 2 号食品方向开始不断绕圈，如果 Diana 走到了食品 i ，并且食品 i 的和下一个食品 j 的辣度有 $\gcd(i, j) = 1$ 的关系时，Diana 就把食品 j 给吃了，此时 i 的下一个食品就变成了 j 的下一个食品，然后 Diana 会直接走到 i 的下一个食品处。Diana 会持续绕圈直到没有东西吃为止。

为了摸清一下食物的辣度，Diana 找到你，请你帮忙按被吃的先后顺序打印出 Diana 能吃到的食品的编号。因为她们会吃 T ($1 \leq T \leq 10^4$) 次烧烤，所以你要打印 T 次食物清单。

$\gcd(x, y)$ 是 x 和 y 的最大公因数。

Input

第一行包含一个整数 T ($1 \leq T \leq 10^4$)，代表吃烧烤的次数。

每次吃烧烤包含两行：

- 第一行包含一个整数 N ($1 \leq N \leq 2 \times 10^5$)，代表烧烤食品数量；
- 第二行包含 N 个整数 $a_1, a_2, a_3, \dots, a_n$ ，其中 a_i ($1 \leq a_i \leq 10^9$) 代表第 i 个食品的辣度。

output

对于每次吃烧烤输出一行，先输出能吃到的食品数量，按食品被吃的先后顺序输出食品编号，中间用空格隔开。

Example

Input


```
5
5
5 9 2 10 15
6
1 2 4 2 4 2
2
1 2
1
1
1
1
2
```

Output

```
2 2 3
2 2 1
2 2 1
1 1
0
```

Note

对于第一次吃烧烤，*Diana* 首先在 1 号食品位置，由于 $\gcd(5, 9) = 1$ 因此 *Diana* 吃掉 2 号食品并到达 3 号食品位置；由于 $\gcd(2, 10) = 2$ ，*Diana* 没吃东西并到了 4 号食品位置；由于 $\gcd(10, 15) = 5$ ，*Diana* 没吃东西并到了 5 号食品位置；由于 $\gcd(15, 5) = 5$ ，*Diana* 没吃东西并到了 1 号食品位置；由于 $\gcd(5, 2) = 1$ 因此 *Diana* 吃掉 3 号食品并到达 4 号食品位置；之后 *Diana* 没有东西吃了。因此输出食品清单“2 2 3”。

Ⅷ. 欢迎来到 *Gensokyo*



Diana、*Nanami* 等人进入了众神眷恋的 *Gensokyo*。要想进入 *Gensokyo*，必须突破 *Reimu* 的弹幕。*Reimu* 将带领她的弹幕阻止你进入 *Gensokyo*。

Reimu 想组织他的弹幕作为一种符卡。这有 N 个弹幕可以使用，并且这个符卡所包含的弹幕必须是可用弹幕的一个子集。但是有一些弹幕如果一起选择的话是会发生冲突的，因此 *Reimu* 不会同时选择两个互相冲突的弹幕。并且 *Reimu* 不希望弹幕太多或者太少，太多发动符卡消耗的能量较多，太少又无法阻止入侵者的进入。更具体地说，可以选择弹幕 i 当且仅当**最终符卡包含的弹幕数量**是不小于 l_i 并且不大于 r_i 的。

为了进入 *Gensokyo* 你需要提前预知一下 *Reimu* 会如何选择弹幕，即计算有 *Reimu* 能组成多少种不同的符卡，两个符卡不同当且仅当两个符卡包含的弹幕不同。因为答案太大了，你需要对答案取 998244353 的模数。

Input

第一行包含两个整数 N, M ($1 \leq N \leq 3 \times 10^5, 0 \leq m \leq \min(20, \frac{n(n-1)}{2})$)，分别表示弹幕的数量和发生冲突弹幕对的个数；

接下来的 N 行，每行包含两个整数 l_i 和 r_i ($1 \leq l_i \leq r_i \leq n$)，意义见上文；

接下来的 M 行，每行包含两个整数 a_i 和 b_i ($1 \leq a_i \leq b_i \leq n$)，表示第 a_i 的弹幕和第 b_i 的弹幕会发生冲突。

Output

输出合法方案的数量，答案对 998244353 取模。

Example

Input

```
3 0
1 1
2 3
1 3
```

Output

```
3
```

Input

```
3 1
1 1
2 3
1 3
2 3
```

Output

```
2
```

II. *Yukari* 修建结界



Yukari 自从 17 岁就开始修建结界了，100 岁也在修，200 岁也在修，300 岁也在修。但是现在终于要完工了。

结界可以表示为 N 个整数组成的序列 a ， a_i 指的是结界第 i 个部分的高度。*Yukari* 只能使用一种大小为 2×1 的封印去修建结界，封印可以使用无数次。*Yukari* 能水平的放置封印，更具体地说，如果存在 $1 \leq i < N$ ，使得当前高度 $a_i = a_{i+1}$ ，的话 *Yukari* 就可以把一个封印放置 i 和 $i+1$ 的上面，使得 i 和 $i+1$ 位置上的高度都增加 1。当然 *Yukari* 也能竖直放置封印，也就是说，能使得任意一个 i 位置的高度增加 2。

Yukari 是一个完美主义者，她想知道结界什么时候完成，当：

1. 所有的位置 i 具有相同的高度；
2. 结界没有任何的空隙。

Yukari 想知道她是否能完成结界，通过使用任意次的封印（可能为 0 次）。

Input

第一行包含一个整数 N ($1 \leq N \leq 2 \times 10^5$)，表示结界有多少个部分。

第二行包含 N 个整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$)，每个部分的初始化高度。

Output

如果 *Yukari* 能完成结界，请输出“YES”，否则输出“NO”（均无引号）。

Example

Input

```
5
2 1 1 2 5
```

Output

YES

Input

```
3
4 5 3
```

Output

YES

Input

```
2
10 10
```

Output

YES

Input

```
3
1 2 3
```

Output

NO

Note

在第一个样例中，可以在 2 和 3 位置上放置一个水平封印使得 $a = [2, 2, 2, 2, 5]$ ，然后放置 3 个水平封印在位置 1 和 2 上，最后放置 3 个水平封印在位置 3 和 4 上，此时 $a = [5, 5, 5, 5, 5]$ 。

在第二个样例中，在 位置 3 放置一个竖直封印，在 2 3 上放置一个水平封印，最后在 1 上放置一个竖直封印即可。

在最后一个样例中无论怎样放置都无法满足要求。

M. *Flandre* 的简单题



Flandre 天天被关在地下室里非常的无聊，于是她想让你帮助她解决一道 *Remilia* 交给她的一道简单题，解决了之后她便能去外面找你和你一起玩游戏。

Flandre 有两个偶数 x 和 y 。请帮助她找到一个整数 n ($1 \leq n \leq 2 \times 10^{18}$)，使得 $n \bmod x = y \bmod n$ 。在这里， $a \bmod b$ 指的是 a 除以 b 的余数。如果有多个答案，输出任意一个即可，可以证明的是，答案一定存在。

Input

第一行包含一个整数 T ($1 \leq T \leq 2 \times 10^5$)，表示测试集的数量。

每一个测试集包含一行两个整数 x 和 y ($2 \leq x, y \leq 10^9$)，保证两个整数都是偶数。

Output

对于每个测试集，输出一个整数 n ($1 \leq n \leq 2 \times 10^{18}$)，要求 n 满足上述条件，如果有多个答案，输出任意一个即可，可以证明的是，答案一定存在。

Example

Input

```
4
4 8
4 2
420 420
69420 42068
```

Output

```
4
10
420
9969128
```

Note

在第一个样例中, $4 \bmod 4 = 8 \bmod 4 = 0$;

在第二个样例中, $10 \bmod 4 = 2 \bmod 10 = 2$;

在第三个样例中, $420 \bmod 420 = 420 \bmod 420 = 0$ 。

N. *Reimu* 的城管任务



女仆长 *Sakuya* 觉得红魔馆距离神社的距离实在太远了, 于是她找到 *Reimu* 想重新规划幻想乡的道路。

幻想乡的道路交通系统是一颗带有 N 个节点的无向树。一颗无向树是指一个有 $N - 1$ 条边的无向连通图。

Reimu 现在正在治退异变, 你的任务是帮助城管 *Reimu* 重新规划道路。你需要加入最少数量的边使得从节点 1 到任意一个节点的最短距离最多为 2。需要注意的是, 你不能加入自环或者重边。

Input

第一行包含一个整数 N ($2 \leq N \leq 2 \times 10^5$)，表示树上顶点的数量。

下面 $N - 1$ 行每一行包含两个数字 u_i 和 v_i ($1 \leq u_i, v_i \leq N$)，表示节点 u_i 和 v_i 之间连有一条边。保证输入的数据合法，即输入的图是一颗树且没有自环或重边。

Output

输出仅一行数字，你添加的最少边的数目。保证答案满足加入最少数量的边使得从节点 1 到任意一个节点的最短距离最多为 2，并且不能加入自环或者重边。

Example

Input

```
7
1 2
2 3
2 4
4 5
4 6
5 7
```

Output

```
2
```

Input

```
7
1 2
1 3
2 4
2 5
3 6
1 7
```

Output

```
0
```

Input

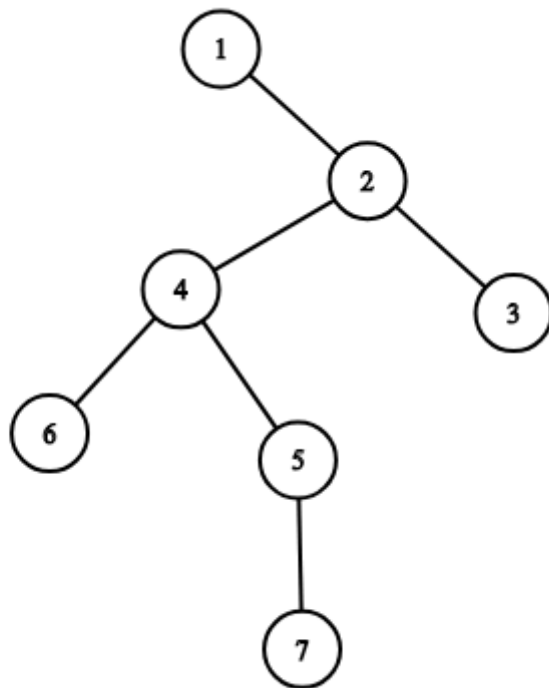
```
7
1 2
2 3
3 4
3 5
3 6
3 7
```

Output

1

Note

在第一个样例中，输入的树为：



答案为 2，可能的添加边的方案为 $[(1, 5), (1, 6)], [(1, 4), (1, 7)], [(1, 6), (1, 7)]$

①. Marisa 的魔法棋盘



一天，*Reimu* 和 *Marisa* 在魔法森林里玩棋。但是她们觉得普通的棋子游戏太无聊了，所以她们决定自己设计一套游戏规则。

魔法棋盘是一个正方形网格，“魔法蘑菇”作为一颗棋子，能够攻击它所在单元格上水平、竖直、以及两个对角线四个方向上的所有敌方棋子。如果一个敌方棋子和“魔法蘑菇”处在同一水平线、竖直线或者对角线上，那么“魔法蘑菇”可以立即跳到敌方棋子的位置上并将敌方棋子消灭。但是，如果“魔法蘑菇”和敌方棋子之间存在其他棋子，那么“魔法蘑菇”就没法跳过去消灭敌方棋子。

现在有一个大小为 $N \times N$ 的魔法棋盘，我们记第 r 行第 c 列所在的单元格为 (r, c) （左上角为 $(1, 1)$ ，右下角为 (n, n) ）。*Reimu* 的“魔法蘑菇”在 $(1, 1)$ 的位置上，而 *Marisa* 的“魔法蘑菇”在 $(1, n)$ 的位置上。除了这两个位置的所有其他的单元格都包含一个中立的“蘑菇士兵”棋子，不属于任何玩家。

Reimu 和 *Marisa* 依次轮流进行游戏，*Reimu* 作为先手进行开局。

每一步当前玩家都可以选择她的“魔法蘑菇”去消灭“蘑菇士兵”或者敌方的“魔法蘑菇”。如果一个玩家无法进行移动即四个方向均没有“蘑菇士兵”或者敌方的“魔法蘑菇”时或者自己的“魔法蘑菇”被敌方的“魔法蘑菇”消灭了，那么这个玩家将输掉本次的对局。

现在 *Reimu* 想知道如果双方都采取最优策略的话这场对局的赢者是谁，快来帮助 *Reimu*，城管 *Reimu* 可不想输给 *Marisa*。

Input

第一行仅包含一个整数 N ($2 \leq N \leq 10^9$)，表示棋盘的大小。

Output

第一行包含一个字符串“white”或者“black”，如果双方都采取最优策略的最终赢者，因为 *Marisa* 只喜欢黑色和白色。

如果答案是“white”即 *Reimu* 赢了这局对局，你应该在第二行输出两个整数 r 和 c 表示单元格 (r, c) ，*Reimu* 开局第一步应该将她的“魔法蘑菇”移动到的下一个单元格。如果有很多答案满足要求，输出 r 最小的那个答案，如果还有很多答案那么输出 c 最小的答案。

Example

Input

```
2
```

Output

```
white
1 2
```

Input

```
3
```

Output

```
black
```

Note

在第一个样例中 *Reimu* 的“魔法蘑菇”在 $(1, 1)$ 可以直接一步消灭 *Marisa* 的“魔法蘑菇”，所以 *Reimu* 是最终赢者，在第一行输出“white”。并且 *Reimu* 第一步走到 $(1, 2)$ 是最优选择，因此在第二行输出 $1\ 2$ 。

对于第二个样例，可以证明双方都采取最优策略下，赢者是 *Marisa*。因此输出“black”。